

NAGphormer : A Tokenized Graph Transformer for node classification in large graphs

CHAE WON PARK

Network Science Lab
Dept. of Artificial Intelligence
The Catholic University of Korea

E-mail: codnjsld@catholic.ac.kr

Contents



- Introduction
 - Limitation of previous work
 - Contributions
- Background
 - Decoupled GCN
 - Graph Transformer
- NAGphormer
 - Hop2Token
 - Framework
 - Theoretical analyses
- Experiments
 - Comparison of Datasets
 - Ablation Study
 - Parameter Study

Limitation of previous work

- Message passing-based GNNs have a number of inherent limitations, **including over-smoothing and over-squashing with the increment of model depth**.
- Existing Graph Transformers have developed techniques to address this issue.
- By integrating structural information into the mode, GT exhibit competitive performance on various graph mining tasks.
- But existing GT treat the nodes as independent tokens and struct a single sequence composed of all the node tokens. Mini-batch training is unsuitable for GT because using a single long sequence as the input.
- Unlike existing GT that regard the nodes as independent tokens, NAGphormer treats each node as a sequence and constructs tokens for each node by Hop2Token module.
- By treating **each node as a sequence of tokens**, NAGphormer could be trained in a mini-batch manner and hence can **handle large graphs**.

Contributions

- Propose Hop2Token module
 - Hop2Token aggregates the neighborhood features **from each hop into a node representation**, resulting in a sequence of token vectors that preserves neighborhood information for different hops.
 - In this way, we can regard each node in the complex graph data as a sequence of tokens.
- Propose NAGphormer model
 - It's for node classification task.
 - Trained in a **mini-batch manner** and enables the model to **handle large graphs**.
 - Attention-based readout function to adaptively learn the importance of different-hop neighborhoods
- Outperforms
 - NAGphormer can learn more expressive node representations from the multi-hop neighborhoods compared to decoupled GCN.
 - NAGphormer outperforms existing GT and mainstream GNNs in small to large dataset.

Decoupled GCN

- GCN
 - Coupled design would lead to the over-smoothing problem when the number of layers increases, limiting the model to capture deep structural information.

$$\mathbf{H}^{(l+1)} = \sigma(\hat{\mathbf{A}}\mathbf{H}^{(l)}\mathbf{W}^{(l)})$$

- Decoupled GCN
 - Separates the feature transformation and neighborhood aggregation in the GCN layer and treats them as independent modules.
 - Decoupled design exhibits high computational efficiency and enables the model capture deeper structural information.

$$\mathbf{Z} = \sum_{k=0}^K \beta_k \mathbf{H}^{(k)}, \mathbf{H}^{(k)} = \hat{\mathbf{A}}\mathbf{H}^{(k-1)}, \mathbf{H}^{(0)} = f_{\theta}(\mathbf{X})$$

Final representation	aggregation	Hidden representation	Neural Network module	Feature matrix
-------------------------	-------------	--------------------------	-----------------------------	-------------------

Graph Transformer

- To integrate graph structural information to the Transformer architecture so as to learn the node representations.
 - 1) Replace the positional encoding with Laplacian eigenvectors or degree-related feature vectors to capture the structural features of nodes.
 - 2) In addition to positional encoding, GNNs are used as auxiliary modules to enable the Transformer model to capture structural information.
 - 3) Introduce graph information bias into the attention score of each node pair.

Hop2Token

- To handle graphs at scale, first introduce a novel neighborhood aggregation module called Hop2Token.
- To inherit the desirable properties, Hop2Token designed considering the neighborhood information of different hops.

$$\mathbf{x}_v^k = \phi(\mathcal{N}^k(v))$$

Neighborhood embedding Aggregation operator K-hop neighborhood

$$\mathcal{S}_v = (\mathbf{x}_v^0, \mathbf{x}_v^1, \dots, \mathbf{x}_v^K)$$

Sequence

Hop2Token

- Advantages
 - Non-parametric method : It can be conducted offline before model training, and the output of Hop2Token supports mini-batch training -> handle large-scale graphs
 - Helpful for capturing the hop-wise semantic correlation.

Algorithm 1 The Hop2Token Algorithm

Input: Normalized adjacency matrix $\hat{\mathbf{A}}$; Feature matrix $\mathbf{X}$; Propagation step K

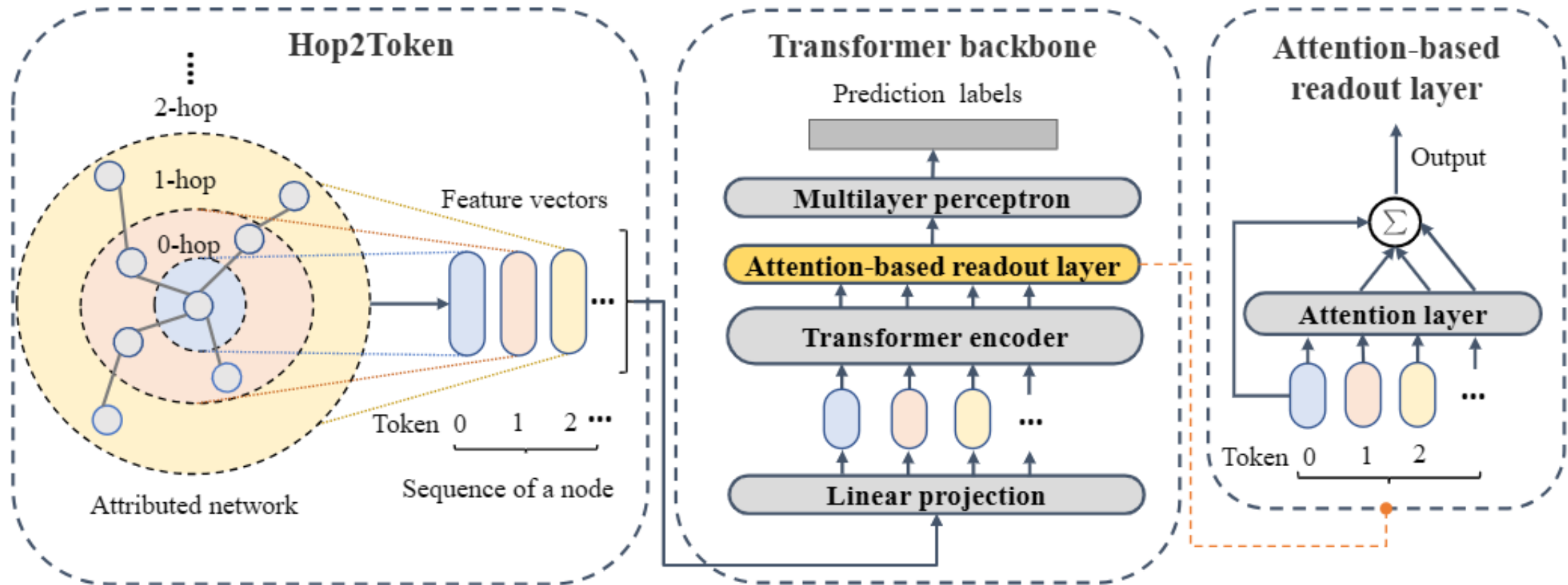
Output: Sequences of all nodes $\mathbf{X}_G$

```
1: for  $k = 0$  to  $K$  do  
2:   for  $i = 0$  to  $n$  do  
3:      $\mathbf{X}_G[i, k] = \mathbf{X}[i];$   
4:   end for  
5:    $\mathbf{X} = \hat{\mathbf{A}}\mathbf{X};$   
6: end for  
7: return Sequences of all nodes  $\mathbf{X}_G$ ;
```

$$\mathbf{X}_k = \hat{\mathbf{A}}^k \mathbf{X}$$

K-hop neighborhood matrix

Framework



Theoretical analyses

- complexity

Time complexity. The time complexity of NAGphormer mainly depends on the self-attention module of the Transformer. So the computational complexity of NAGphormer is $O(n(K + 1)^2 d)$, where n denotes the number of nodes, K denotes the number of hops and d the dimension of parameter matrix (*i.e.*, feature vector).

Space complexity. The space complexity is based on the number of model parameters and the outputs of each layer. The first part is mainly on the Transformer layer $O(d^2 L)$, where L is the number of Transformer layers. The second part is on the attention matrix and the hidden node representations, $O(b(K + 1)^2 + b(K + 1)d)$, where b denotes the batch size. Thus, the total space complexity is $O(b(K + 1)^2 + b(K + 1)d + d^2 L)$.

Theoretical analyses

- NAGphormer could learn more informative node representations from the multi-hop neighborhoods than decoupled GCN does.

- Structural Endoding

$$\mathbf{X}' = \mathbf{X} \parallel \mathbf{U}$$

concat Structural
Encoding

- Attention-based readout function

$$\alpha_k = \frac{\exp((\mathbf{Z}_0 \parallel \mathbf{Z}_k) \mathbf{W}_a^\top)}{\sum_{i=1}^K \exp((\mathbf{Z}_0 \parallel \mathbf{Z}_i) \mathbf{W}_a^\top)}$$

K-hop
representation

$$\mathbf{Z}_{out} = \mathbf{Z}_0 + \sum_{k=1}^K \alpha_k \mathbf{Z}_k$$

Comparison of Datasets – small scale

Table 1: Comparison of all models in terms of mean accuracy $\pm$ stdev (%) on small-scale datasets. The best results appear in **bold**. OOM indicates the out-of-memory error.

Method	Pubmed	CoraFull	Computer	Photo	CS	Physics
GCN	86.54 ± 0.12	61.76 ± 0.14	89.65 ± 0.52	92.70 ± 0.20	92.92 ± 0.12	96.18 ± 0.07
GAT	86.32 ± 0.16	64.47 ± 0.18	90.78 ± 0.13	93.87 ± 0.11	93.61 ± 0.14	96.17 ± 0.08
APPNP	88.43 ± 0.15	65.16 ± 0.28	90.18 ± 0.17	94.32 ± 0.14	94.49 ± 0.07	96.54 ± 0.07
GPRGNN	89.34 ± 0.25	67.12 ± 0.31	89.32 ± 0.29	94.49 ± 0.14	95.13 ± 0.09	96.85 ± 0.08
GraphSAINT	88.96 ± 0.16	67.85 ± 0.21	90.22 ± 0.15	91.72 ± 0.13	94.41 ± 0.09	96.43 ± 0.05
PPRGo	87.38 ± 0.11	63.54 ± 0.25	88.69 ± 0.21	93.61 ± 0.12	92.52 ± 0.15	95.51 ± 0.08
GRAND+	88.64 ± 0.09	71.37 ± 0.11	88.74 ± 0.11	94.75 ± 0.12	93.92 ± 0.08	96.47 ± 0.04
GT	88.79 ± 0.12	61.05 ± 0.38	91.18 ± 0.17	94.74 ± 0.13	94.64 ± 0.13	97.05 ± 0.05
Graphormer	OOM	OOM	OOM	92.74 ± 0.14	OOM	OOM
SAN	88.22 ± 0.15	59.01 ± 0.34	89.83 ± 0.16	94.86 ± 0.10	94.51 ± 0.15	OOM
GraphGPS	88.94 ± 0.16	55.76 ± 0.23	OOM	95.06 ± 0.13	93.93 ± 0.12	OOM
NAGphormer	89.70 ± 0.19	71.51 ± 0.13	91.22 ± 0.14	95.49 ± 0.11	95.75 ± 0.09	97.34 ± 0.03

Comparison of Datasets – large scale

Table 2: Comparison of all models in terms of mean accuracy $\pm$ stdev (%) on large-scale datasets. The best results appear in **bold**.

Method	AMiner-CS	Reddit	Amazon2M
PPRGo	49.07 $\pm$ 0.19	90.38 $\pm$ 0.11	66.12 $\pm$ 0.59
GraphSAINT	51.86 $\pm$ 0.21	92.35 $\pm$ 0.08	75.21 $\pm$ 0.15
GRAND+	54.67 $\pm$ 0.25	92.81 $\pm$ 0.03	75.49 $\pm$ 0.11
NAGphormer	56.21 $\pm$ 0.42	93.58 $\pm$ 0.05	77.43 $\pm$ 0.24

Ablation Study – Structural encoding

Table 3: The accuracy (%) with or without structural encoding.

	Pubmed	CoraFull	CS	Computer	Photo	Physics	Aminer-CS	Reddit	Amazon2M
W/O-SE	89.06	70.42	95.52	90.44	95.02	97.10	55.64	93.47	76.98
With-SE	89.70	71.51	95.75	91.22	95.49	97.34	56.21	93.58	77.43
Gain	+0.64	+1.09	+0.23	+0.78	+0.47	+0.24	+0.57	+0.11	+0.45

Ablation Study - Attention based readout function

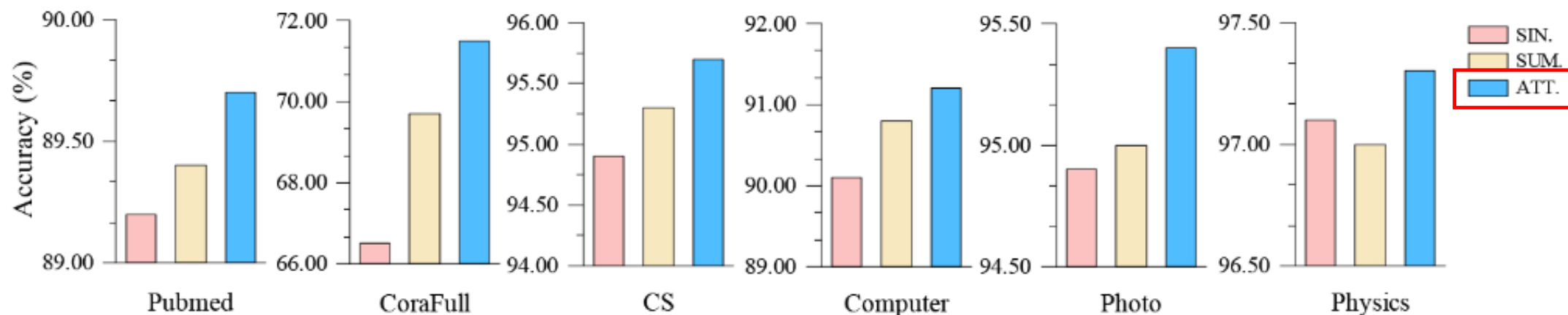
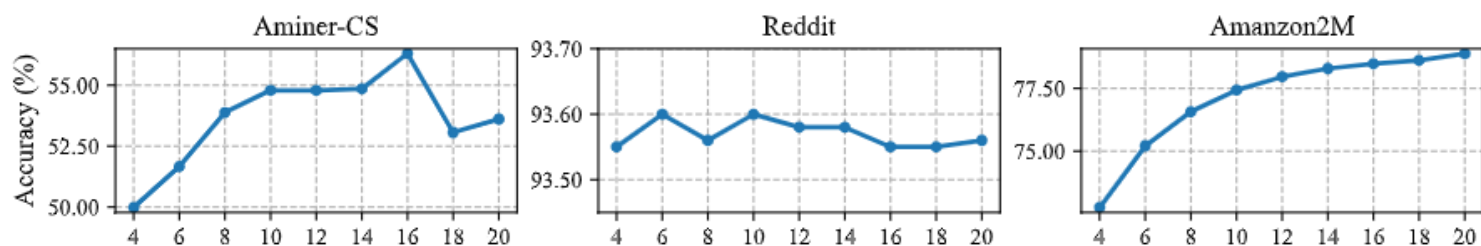
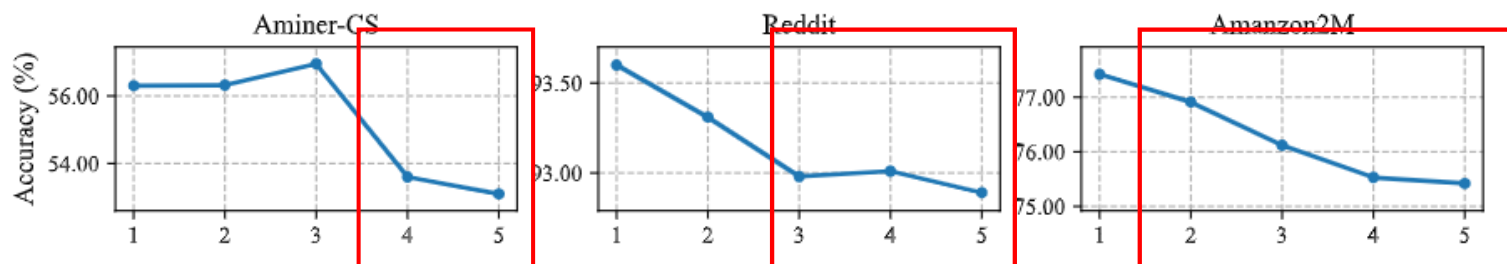


Figure 2: The performance of NAGphormer via different readout functions.

Parameter Study

(a) On the number of propagation steps K (b) On the number of Transformer layers L

overfitting

Figure 3: Performance of NAGphormer on different parameters.

